C# Lab Manual 2: Controls and Text Validation

Objective:

1. This lab manual introduces essential C# controls, such as ComboBox, ListBox, Label, and TextBox.
2. You will also learn how to validate user inputs.

1. Introduction to Controls

Controls are essential UI elements that interact with users. They allow data entry, display data, and handle events. In C#, Windows Forms provides a variety of controls such as TextBox, ComboBox, ListBox, etc.

1.1 ComboBox

The ComboBox control provides a drop-down list of items from which users can choose only one item.

Basic Properties

Property	Description
`Items`	Collection of items in the ComboBox.
`SelectedItem`	Gets or sets the currently selected item.
`SelectedIndex`	Gets or sets the index of the selected item (-1 if none is selected).
`DropDownStyle`	Determines whether the ComboBox is editable (`DropDown` , `Simple` , `DropDownList`).
`Text`	Gets or sets the text of the ComboBox.
`MaxDropDownItems`	Specifies the maximum number of items displayed in the drop-down list.

Creating a ComboBox:

1. Drag the ComboBox from the toolbox onto the form.
2. To populate the ComboBox, use the Items property.

```
comboBox1.Items.Add("Option 1");
comboBox1.Items.Add("Option 2");
comboBox1.SelectedIndex = 0; // Selects first item
```

You can also use an array to fill the ComboBox:

```
string[] items = { "Item A", "Item B", "Item C" };
comboBox1.Items.AddRange(items);
```

- To handle the selected item change event:

```
comboBox1.SelectedIndexChanged += new EventHandler(comboBox1_SelectedIndexChanged);

private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    MessageBox.Show("Selected item: " + comboBox1.SelectedItem.ToString());
}
```

1.2 ListBox

The ListBox control displays a list of items and allows the user to select one or more items.

Basic Properties

Property	Description
`Items`	Collection of items in the ListBox.
`SelectedItem`	Gets or sets the currently selected item.
`SelectedIndex`	Gets or sets the index of the selected item (-1 if none is selected).
`SelectionMode`	Determines whether the user can select one or multiple items (`None`, `One`, `MultiSimple`, `MultiExtended`).
`Text`	Gets or sets the text of the selected item.

Creating a ListBox:

1. Drag the ListBox from the toolbox onto the form.
2. Populate it similarly to the ComboBox:

```
listBox1.Items.Add("Option 1");
listBox1.Items.Add("Option 2");
listBox1.Items.Add("Option 3");
```

Multi-selection:

- Enable multi-selection by setting the SelectionMode property:

```
listBox1.SelectionMode = SelectionMode.MultiExtended;
```

1.3. TextBox

A TextBox allows users to enter or edit text.

Basic Properties:

Property	Description
`Text`	Gets or sets the text inside the TextBox.
`MaxLength`	Defines the maximum number of characters allowed.
`ReadOnly`	If `true`, the TextBox is read-only.
`Multiline`	If `true`, allows multiple lines of text.
`PasswordChar`	Masks input characters (used for password fields).

Example

1. Sets the initial text displayed in the **TextBox** to "Enter name".
2. Limits the number of characters that can be entered in the **TextBox** to **50**.
3. Makes the **TextBox read-only**, meaning the user cannot edit the text inside it.

```
textBox1.Text = "Enter name";
textBox1.MaxLength = 50;
textBox1.ReadOnly = true; // Prevents editing
```

1.4. Label

A Label displays static text in a form.

Basic Properties:

Property	Description
`Text`	The text displayed on the label.
`Font`	Defines the font style and size.
`ForeColor`	Sets the text color.
`BackColor`	Sets the background color.
`AutoSize`	If `true`, resizes automatically based on text length.

Example

1. Add a label control with the following settings:
    a. Set the label text to “Welcome”

b. Set the text color to Blue
c. Set the font type to Arial, font size to 12 and font style to bold.

```
label1.Text = "Welcome!";
label1.ForeColor = Color.Blue;
label1.Font = new Font("Arial", 12, FontStyle.Bold);
```

Lab Task:

1. Create a combo box and a label. Set the combo box items to: Apple, Orange, Banana.
2. **Apple** should be selected by default.
3. When you select another item, the label updates dynamically.

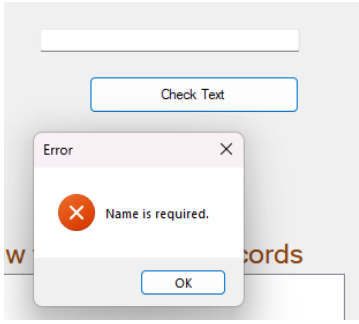
2. Text Validation

Text validation is an essential task for ensuring that user input is correct. You can validate text inputs from controls like TextBox before performing further operations.

2.1 Basic Validation:

Ensure that the user enters the required text.

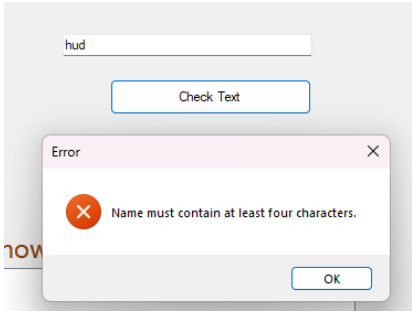
```
if (txtName.Text.Trim() == "")
{
    MessageBox.Show("Name is required.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error)
    return;
}
```



2.2 Length Validation:

Check if the text has a certain length.

```
if (TxtName.Text.Length < 4)
{
    MessageBox.Show("Name must contain at least four characters.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}
```



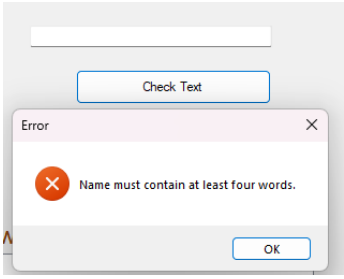
2.3 Custom Validation (Multiple Words):

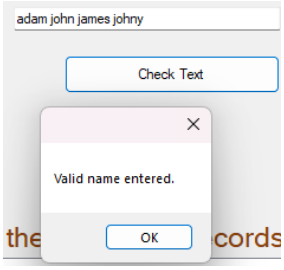
Ensure the text contains a minimum number of words (for example, 4 words).

```
var words = TxtName.Text.Trim().Split(new char[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);

if (words.Length < 4)
{
    MessageBox.Show("Name must contain at least four words.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

MessageBox.Show("Valid name entered.");
```





Lab Task:

1. Each word must be at least **3 characters long**.
2. The input must contain **at two words**.
3. Display appropriate error messages using a **MessageBox** if the conditions are not met.